CONTEXT ROT FIREWALL

Adaptive Context Management for Long-Running AI Agents

Final-Year B.Tech CSE Major Project

Project summary, proposed architecture, implementation roadmap, and evaluation guidelines

Selected direction: Build a model-agnostic middleware layer that adapts context-management decisions to the current task, token budget, information importance, retrieval confidence, and measured overhead.

Planning document • Version 1.0 • October 2026

1. Project Summary

Project title: Context Rot Firewall: Adaptive Context Management for Long-Running AI Agents

Problem: As an AI agent works through a long task, its context can accumulate file contents, tool results, old decisions, and irrelevant information. Existing compaction and memory mechanisms help, but may not always preserve the information needed for the next step.

Proposed solution: A model-agnostic middleware layer that dynamically selects a context-management strategy according to the current sub-goal, remaining token budget, information importance, retrieval confidence, and measured overhead.

Research objective: Determine whether adaptive context management improves task success and information retention compared with ordinary compaction and fixed relevance-based policies under comparable token budgets.

The project should not claim that context rot is solved universally. Its contribution is a clearly defined adaptive policy, a reusable implementation, and a reproducible evaluation.

2. Proposed System Architecture

The firewall runs before each LLM request. The agent and LLM remain replaceable; the firewall manages the context that the application controls.

Layer	Responsibility
1. AI agent / orchestrator	Plans the task, calls tools, receives observations, and decides the next step.
2. Context interceptor	Captures the message history and tool outputs before each model request.
3. Context analyzer	Tracks the current sub-goal, chunks, token budget, dependencies, and protected information.
4. Adaptive policy engine	Chooses whether eligible chunks should be kept, compressed, archived, or retrieved.
5. Context assembler	Builds valid, budget-aware model input while preserving roles, order, and tool-call relationships.
6. LLM and tools	Generate responses or tool calls; tool results return to the agent loop.
7. Archive and telemetry	Stores original chunks and embeddings; records decisions, token counts, latency, and outcomes.
8. Dashboard	Displays context health and benchmark results; observes the system without controlling the agent.

3. Adaptive Decision-Making

The system should begin with a rule-based, explainable policy. It does not need a trained ML policy at the start. A learned policy should only be considered if experiments show that it adds value.

Signals the policy should consider

• **Relevance to the current sub-goal:** whether a chunk is useful for the current step.

• **Criticality:** whether losing the information could violate a requirement or break a dependency.

• **Recency:** whether the information still reflects the current state.

• **Token cost:** how much context space the chunk consumes.

• **Retrieval confidence:** how reliably the original can be recovered later.

• **Recent outcomes:** whether earlier compression or retrieval decisions helped or harmed task progress.

Example: debugging a failing test

• **Keep:** the original requirement, current bug location, and latest failing test output.

• **Compress:** repetitive directory listings into a concise note containing useful discoveries.

• **Archive:** full outputs from earlier, unrelated test runs while preserving original content and stable identifiers.

• **Retrieve:** an earlier error trace when a new failure indicates it may be relevant again.

Important design rule: Semantic similarity is not the same as importance. Protect system instructions, constraints, unresolved tool interactions, and essential dependencies from ordinary eviction. Never permanently discard content in the initial version.

4. Runtime Workflow

1. Capture. Collect the current task/sub-goal, message history, tool results, roles, and metadata.

2. Chunk. Group related messages and tool outputs into manageable units with stable IDs.

3. Analyze. Estimate tokens and score chunk relevance; identify protected instructions and dependencies.

4. Choose policy. Select keep, compress, archive, or retrieve for eligible chunks.

5. Apply changes. Compress selected content, archive originals, and retrieve prior information when needed.

6. Assemble. Construct valid model input within the configured token budget.

7. Execute. Send the request to the LLM and let the agent execute any returned tool call.

8. Measure. Record task progress, success/failure, token use, latency, and retrieval outcomes; use feedback to tune policy parameters.

5. Recommended Technology Stack

Component	Technology	Purpose
Language	Python	Middleware and agent integration
Initial agent	Custom tool-calling loop	Simple, controllable baseline
Local LLM	Ollama + a small compatible model	Local development without API charges
Embeddings	sentence-transformers / all-MiniLM-L6-v2	Semantic relevance scoring
Archive retrieval	ChromaDB + durable original-content storage	Search and recover archived chunks
Dashboard	Streamlit	Context health and benchmark display
Testing	pytest	Unit and integration tests

6. Suggested Project Directory

```
context-rot-firewall/
■■■ agent/
■   ■■■ runner.py
■   ■■■ tools.py
■■■ firewall/
■   ■■■ interceptor.py
■   ■■■ chunker.py
■   ■■■ analyzer.py
■   ■■■ relevance.py
■   ■■■ adaptive_policy.py
■   ■■■ compressor.py
■   ■■■ storage.py
■   ■■■ retrieval.py
■   ■■■ assembler.py
■■■ evaluation/
■   ■■■ tasks.py
■   ■■■ benchmark.py
■   ■■■ metrics.py
■■■ dashboard/
■   ■■■ app.py
■■■ tests/
■■■ config.yaml
■■■ README.md
```

This is a proposed structure, not existing code. Implement modules incrementally and keep each one independently testable.

7. Implementation Roadmap

Phase 1 — Foundation

Build a controllable baseline agent that reads files, executes predefined tools, receives results, and repeats. Log every model request and tool output.

Phase 2 — Core engine

Implement the interceptor, chunking, stable IDs, token counting, sub-goal tracking, embeddings, and protected-context rules.

Phase 3 — Adaptive policy

Add fixed thresholds first, then budget-aware and task-aware rules. Log why each action was chosen.

Phase 4 — Recovery

Implement compression, original-content storage, ChromaDB retrieval, and valid context reconstruction.

Phase 5 — Evaluation

Track task success, lost requirements, failed retrievals, token use, and latency. Compare adaptive decisions against fixed policies.

Phase 6 — Demonstration

Add the Streamlit dashboard and, if time allows, a second model or agent-framework integration.

8. Evaluation Plan

Evaluate four configurations on the same task set and under comparable token budgets:

Configuration	Strategy
A — Baseline	Agent's normal context handling
B — Simple compaction	Fixed-window truncation or basic summarization
C — Fixed relevance firewall	Relevance scoring with fixed thresholds
D — Adaptive firewall	Task- and budget-aware context policy with feedback tuning

Test scenarios

• **Forgotten requirement:** introduce an early constraint, add unrelated tool calls, then test whether the agent still follows the constraint.

• **Recovering an earlier discovery:** make an early finding relevant again after many intervening steps.

• **Multi-file coding task:** edit several files, run tests, handle failures, and track decisions across many turns.

Metrics

• Task success rate and test pass rate.

• Critical-information retention and lost requirements.

• Input tokens consumed and compression ratio.

• End-to-end latency and extra cost.

• Retrieval accuracy and recovery success.

Hypothesis: An adaptive context-management policy can improve long-running agent reliability over fixed strategies under a comparable token budget, without introducing unacceptable latency or cost. This is a hypothesis to test, not a result to assume.

9. Scope Boundaries and Risks

• Do not train a new LLM for the initial version.

• Do not build a complete agent framework; build only a minimal baseline loop.

• Do not promise compatibility with every commercial agent. Support only interfaces that expose the context needed for management.

• Do not add reinforcement learning before a rule-based adaptive policy has been evaluated.

• Do not claim the firewall solves all context rot or guarantees that important information will never be lost.

• Keep original chunks recoverable; treat the vector database as an index rather than the sole source of truth.

• Preserve message roles, ordering, tool-call IDs, and required instruction structure.

10. First Milestone

Before implementing the firewall, build a baseline agent that completes one small coding task through repeated tool calls. Save its message history and log how the context grows. This provides a controlled environment for evaluating later context-management strategies.

Definition of done: the baseline can run a task end-to-end, every model request and tool result is logged, the task outcome can be checked, and the same task can be rerun under the same conditions.

11. Final Recommendation

Keep the Context Rot Firewall project, but define its contribution as an adaptive, explainable context-management policy evaluated against meaningful baselines. Start with a rule-based system, preserve recoverability, and add more complex learning only if experimental evidence justifies it.

The project will be strongest if it demonstrates not just that context can be reduced, but when adaptive management helps, what trade-offs it introduces, and where it fails.